

Data and Artificial Intelligence

Cyber Shujaa Program

Week 10 Assignment

Deep Learning

Student Name: Violet Joy

Student ID: cs-da01-25025

Table of Content

Data and Artificial Intelligence	1
Cyber Shujaa Program	1
Week 10 Assignment Deep Learning	1
Introduction.....	3
Tasks Completed	4
2.1: Loading and Exploring the Dataset.....	4
2.2: Pre-processing the data	6
2.3: Build the Neural Network model	7
2.4: Compile the Model.....	8
2.5: Train the Model.....	9
2.6: Visualizing training history	11
2.7: Evaluate the Model	13
2.8: Making predictions and analysing performance	15
2.9: Saving and Reloading the model.....	17
Conclusion	19

Introduction

This project focuses on the application of Artificial Neural Networks (ANNs) using TensorFlow and Keras to solve a classic image classification problem with the MNIST dataset. The MNIST dataset, which stands for Modified National Institute of Standards and Technology, is a widely recognized benchmark in the field of machine learning and computer vision. It consists of 70,000 grayscale images of handwritten digits (0–9), each sized 28×28 pixels. These images are labeled into 10 distinct classes, corresponding to the digits from 0 to 9.

The dataset is pre-divided into 60,000 training samples and 10,000 test samples, making it ideal for building, training, validating, and evaluating classification models. The primary objective of this assignment is to design a deep learning model capable of accurately recognizing handwritten digits through a series of well-defined steps that reflect real-world deep learning workflows.

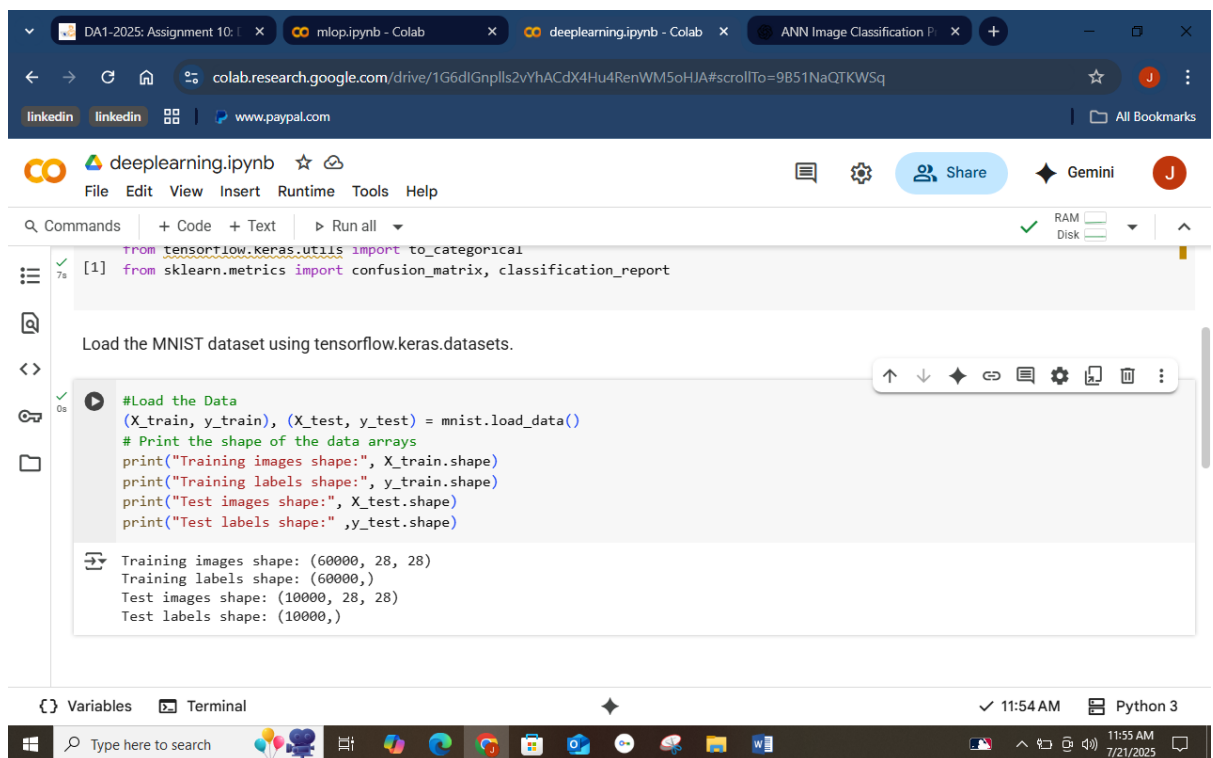
Throughout this project, various core deep learning concepts will be applied. These include preprocessing and visualizing image data, constructing the ANN architecture, compiling and training the model, validating its performance, and evaluating final test accuracy. Additionally, training history will be visualized to assess the model's learning progress over time. Finally, the trained model will be saved and reloaded using Keras' modern model saving format, reinforcing skills related to model persistence and deployment.

Tasks Completed

2.1: Loading and Exploring the Dataset

The MNIST dataset, which is available directly from TensorFlow's Keras API, consists of 70,000 grayscale images of handwritten digits (0 through 9). These are already split into 60,000 training and 10,000 testing samples. Each image is 28x28 pixels and labeled with the correct digit.

- The dataset was loaded using `tensorflow.keras.datasets.mnist.load_data()`.
- The shapes of the training and testing data arrays were printed to verify correct loading.
- To gain an initial understanding of the dataset, 9 random images were visualized using `matplotlib.pyplot`, each accompanied by its label. This step helps confirm that the data is correctly formatted and provides insight into the diversity and quality of the handwriting samples.



```
from tensorflow.keras.utils import to_categorical
[1] from sklearn.metrics import confusion_matrix, classification_report

Load the MNIST dataset using tensorflow.keras.datasets.

#Load the Data
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Print the shape of the data arrays
print("Training images shape:", X_train.shape)
print("Training labels shape:", y_train.shape)
print("Test images shape:", X_test.shape)
print("Test labels shape:", y_test.shape)

Training images shape: (60000, 28, 28)
Training labels shape: (60000,)
Test images shape: (10000, 28, 28)
Test labels shape: (10000,)
```

Fig 2.1.1: Output of shape of data arrays

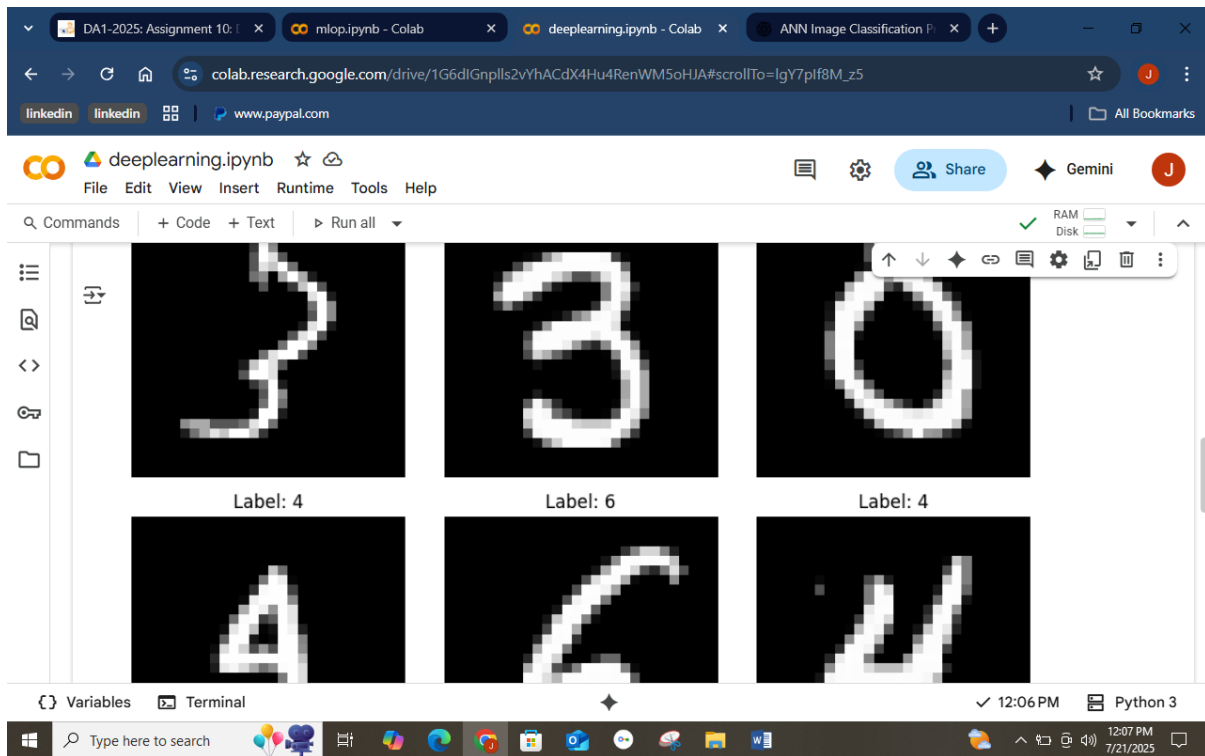


Fig 2.1.2: Output of 9 random images

2.2: Pre-processing the data

Before feeding the images into the neural network, the data must be preprocessed to ensure proper learning:

- **Normalization:** Since the pixel values range from 0 to 255, they were scaled to a [0, 1] range by dividing by 255.0. This helps improve model convergence and stability during training.
- **One-Hot Encoding:** The target labels (0–9) were transformed into one-hot encoded vectors using `tensorflow.keras.utils.to_categorical`. This is necessary because the model outputs a probability distribution across 10 classes, and categorical cross entropy loss requires one-hot encoded labels.
- After preprocessing, the shapes of the data arrays were printed again to confirm that the input and output dimensions align with model expectations.

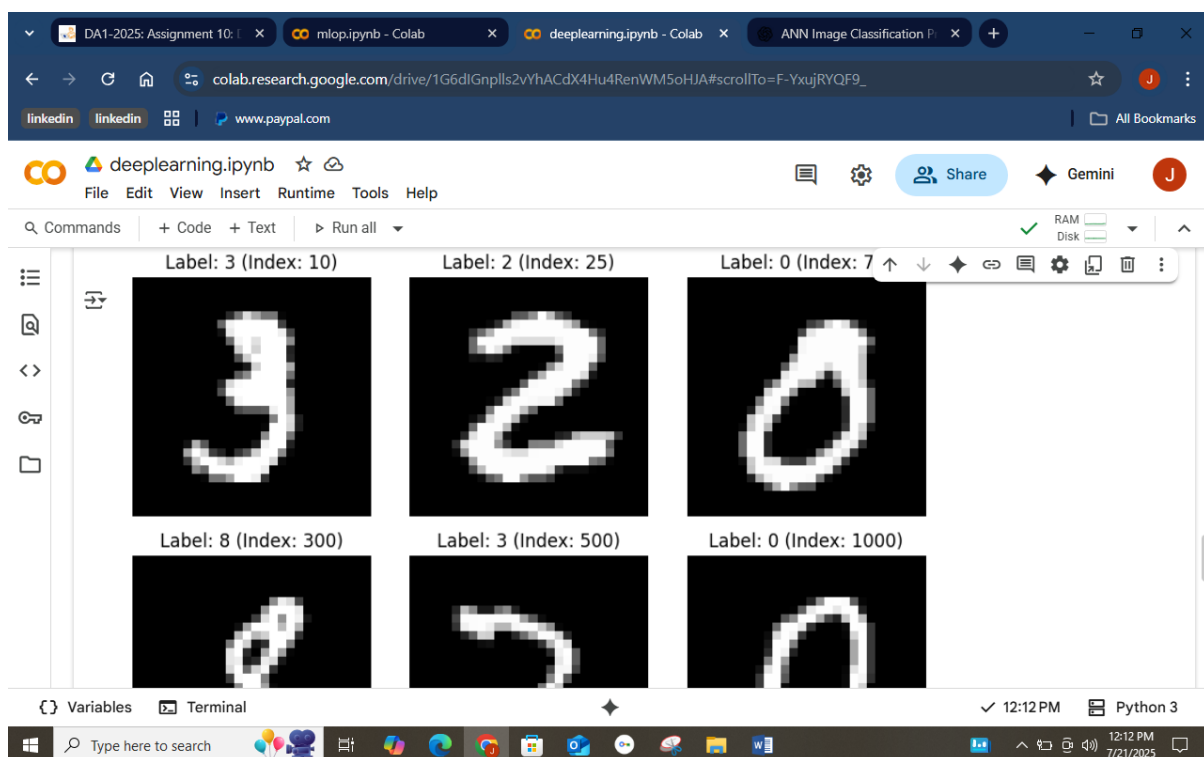


Fig 2.2.1: Output of shape of data arrays

2.3: Build the Neural Network model

A deep learning model was built using the **Sequential API** from Keras, which allows layers to be stacked in a linear fashion.

It includes the following layers:

- **Flatten Layer:** Converts the 2D 28×28 input into a 1D array of 784 pixels, preparing the data for fully connected layers.
- **Hidden Layers:**
 - ❖ First Dense layer with 128 neurons and ReLU activation.
 - ❖ Second Dense layer with 64 neurons and ReLU activation.
 - ❖ These layers allow the model to learn complex patterns from the input data.
- **Dropout Layers:** A dropout rate of 0.3 is used after each hidden layer to randomly deactivate 30% of the neurons during training. This helps prevent overfitting by ensuring that the network does not become too reliant on any specific nodes.
- **Output Layer:** A Dense layer with 10 neurons (one for each digit class) and a softmax activation function. This produces a probability distribution over all classes.

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(10, activation='softmax')
])
```

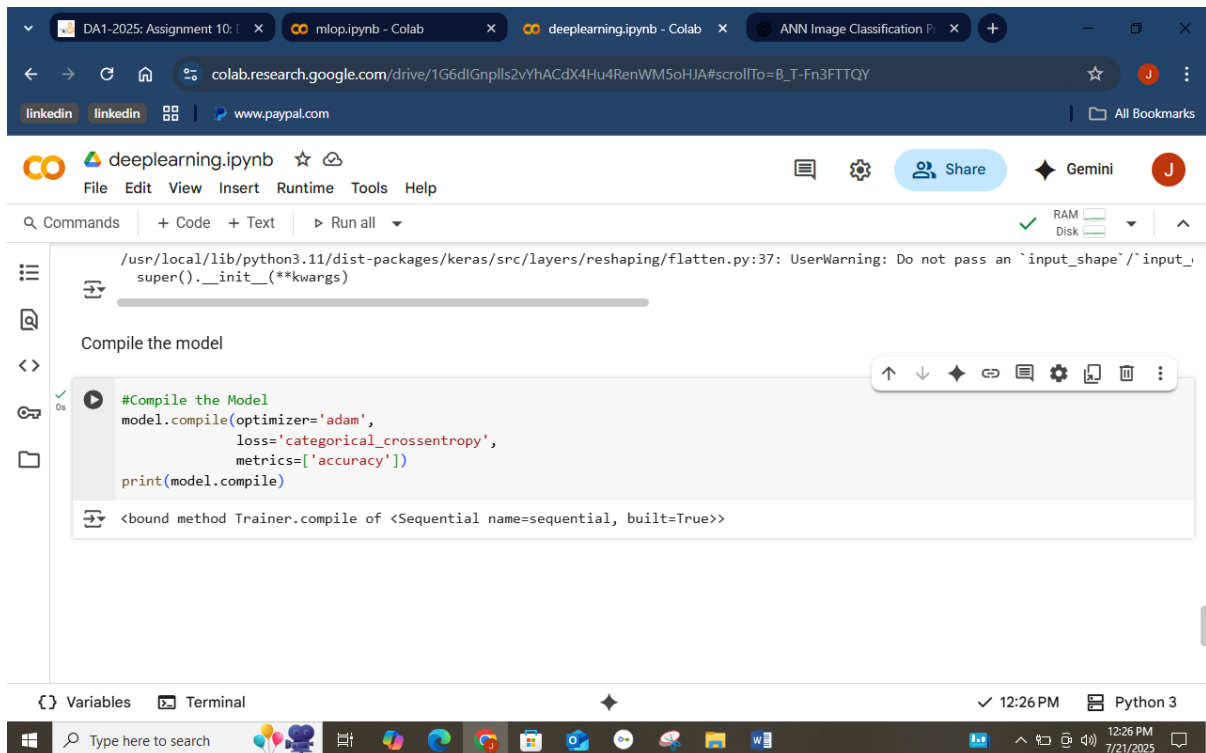
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input\_shape` / `input\_shape` argument to the `Flatten` layer. It will be ignored.

Fig 2.3.1: Output of building the neural network

2.4: Compile the Model

The model was compiled with the following configurations:

- **Optimizer:** Adam optimizer was chosen for its efficiency and adaptive learning rate.
- **Loss Function:** `categorical_crossentropy` was used as it is well-suited for multi-class classification problems with one-hot encoded labels.
- **Evaluation Metric:** Model performance was tracked using `accuracy`, which measures the percentage of correctly classified digits.



```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape` / `input_
super().__init__(**kwargs)
```

Compile the model

```
#Compile the Model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
print(model.compile)
```

<bound method Trainer.compile of <Sequential name=sequential, built=True>>

Fig 2.4.1:Output of compiling the model

2.5: Train the Model

Training is the process where the model learns the relationships between input images and their corresponding labels by adjusting internal parameters (weights and biases) over multiple passes (epochs).

During training:

- The model processes input images in batches.
- It predicts outputs and compares them to the true labels using the loss function.
- The optimizer updates the model's weights using backpropagation and gradient descent, minimizing the loss.
- Over time, the model improves its predictions.

The model was trained using the following parameters:

- `x_train, y_train`: The input images and their one-hot encoded labels.
- `epochs=10`: The model makes 10 full passes over the training data. This is often a good starting point for small-to-medium datasets like MNIST.
- `batch_size=128`: The training data is divided into mini-batches of 128 samples. Training with mini-batches balances performance and stability.
- `validation_split=0.1`: 10% of the training data is set aside for validation. This allows the model's performance to be evaluated on unseen data during training, helping detect overfitting or under fitting early on.
- `history`: The returned object contains logs of the loss and accuracy values at each epoch for both training and validation sets, which can later be used for visualization.

Training Process Summary

In each epoch, the model updates its internal weights to reduce the error between predicted and true labels. With each batch, gradients are computed and weights are updated via the Adam optimizer. Accuracy and loss are recorded for both training and validation data. If the model performs well, training accuracy and validation accuracy both increase while losses decrease.

Monitoring During Training

- During training, a progress bar displays:
- Epoch number: Which training cycle is currently running.
- Loss and accuracy: For both training and validation.

Best Practices During Training

- Monitor both training and validation metrics for signs of overfitting.
- If performance plateaus, consider increasing the number of epochs.
- Use callbacks (like `EarlyStopping`) in more advanced projects to stop training when validation performance stops improving.

By the end of the training process, the model had developed a strong understanding of the digit patterns in the MNIST dataset. This prepares it for final evaluation on the separate test set to confirm its generalization ability.

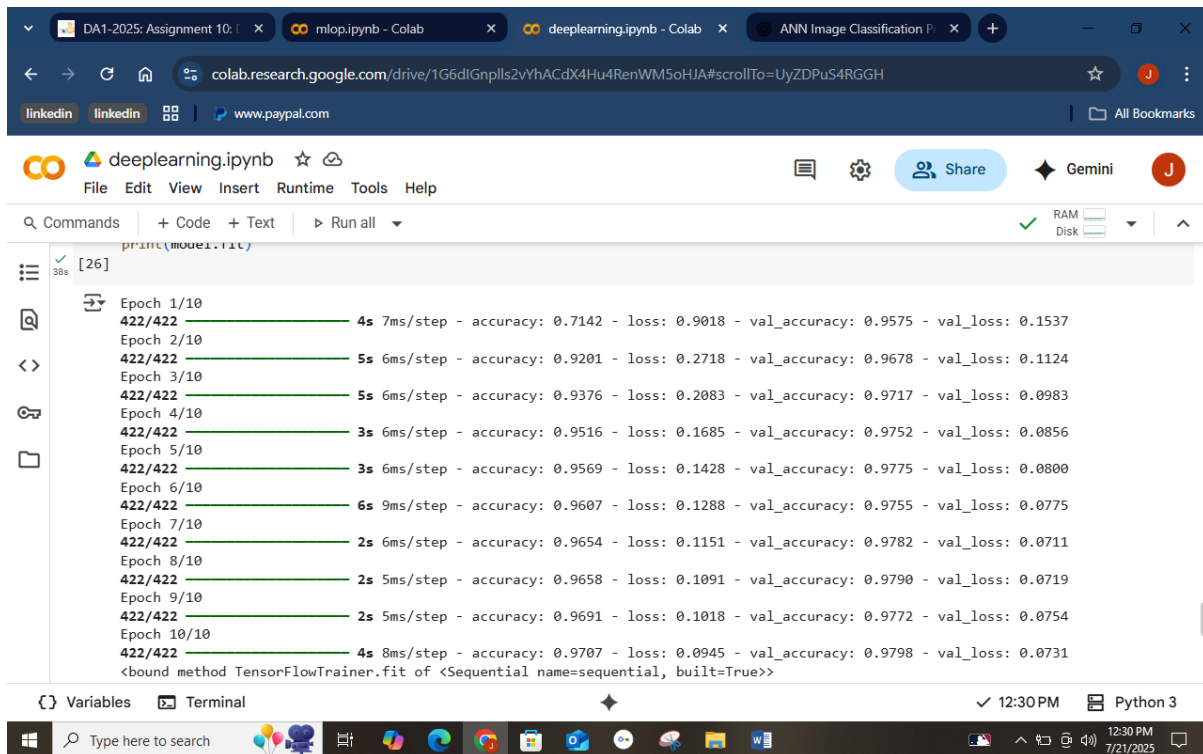


Fig 2.5.1: Output of training the model

2.6: Visualizing training history

Visualizing the model's training history is an essential step in understanding how well the neural network is learning over time. It provides insights into the model's performance during training and helps identify potential issues such as **overfitting**, **under fitting** or **plateauing**.

Purpose of Visualizing Training History

- **Monitor Learning Progress:** Track changes in accuracy and loss across epochs.
- **Detect Overfitting:** Overfitting occurs when the model performs well on training data but poorly on validation data. This is typically seen when validation loss increases while training loss continues to decrease.
- **Optimize Model Architecture:** Based on the learning curves, you can decide whether to modify the model (e.g., add regularization, more data, or layers).
- **Choose Early Stopping:** Visualization helps determine when to stop training to prevent unnecessary epochs once performance stabilizes or worsens.

When training a model using the `fit()` method, Keras returns a **History object** that contains details about the training and validation performance for each epoch. This history object stores training metrics in `history` which is a dictionary with keys like 'loss', 'accuracy', 'val_loss', and 'val_accuracy'.

Interpreting the Plots

- If **training and validation accuracy** both increase steadily and converge, the model is learning well.
- If **validation accuracy plateaus** or **drops** while training accuracy rises, the model may be **overfitting**.
- If both training and validation accuracy stay low, or loss remains high, the model may be **under fitting**, indicating a need for a more complex architecture or longer training.

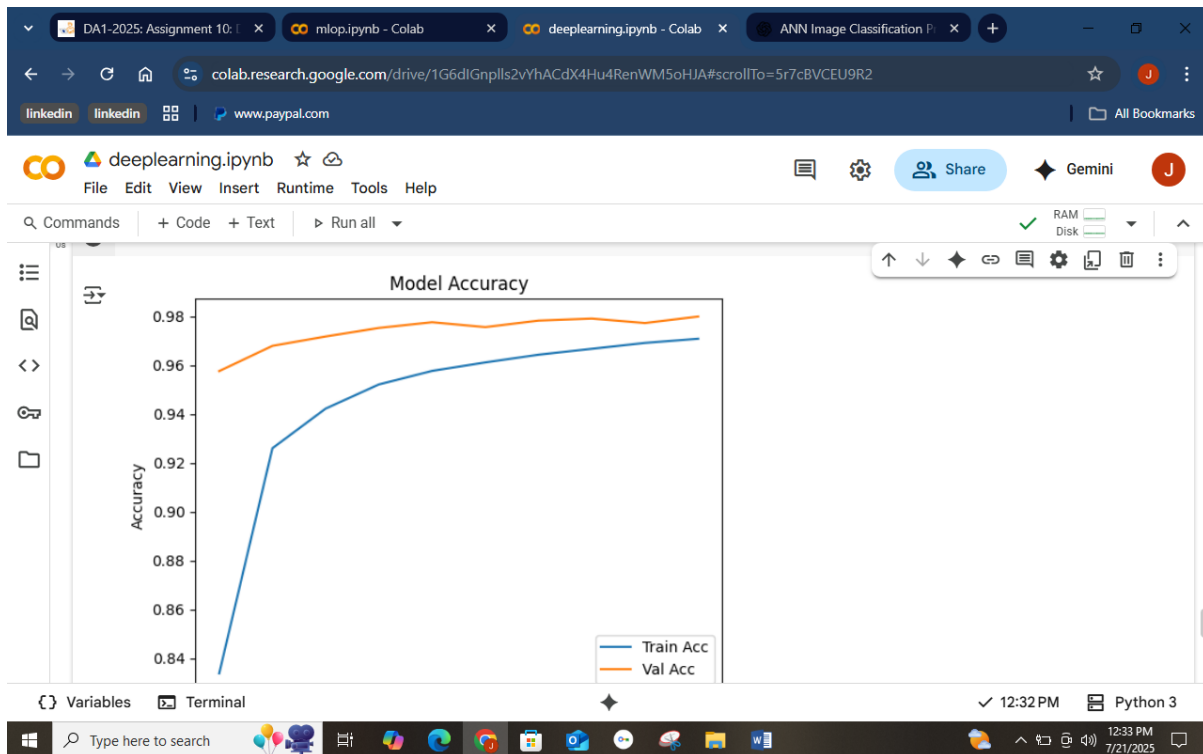


Fig 2.6.1: Output of Epoch against Accuracy

Training Accuracy Increases Steadily:

- Starts at about **84%** and improves consistently to over **96%** by the 10th epoch.
- This indicates that the model is successfully learning patterns from the training data.

Validation Accuracy Starts High and Slightly Improves:

- Starts around **96%**, which is already quite high.
- Peaks near **98%** and fluctuates slightly between **97%–98%** in later epochs.
- This stability suggests that the model generalizes well to unseen data and **is not overfitting**.

Gap Between Training and Validation Accuracy:

- The validation accuracy is consistently higher than training accuracy.
- This is slightly unusual but can happen when:
 - ❖ The validation data is slightly easier.
 - ❖ Dropout or regularization reduces training accuracy but improves generalization

2.7: Evaluate the Model

After training the model, the next critical step is to **evaluate its performance** on completely unseen data in this case, the test set. Evaluation gives a quantitative measure of how well the model generalizes to new inputs, beyond what it saw during training and validation.

In this project, evaluation is done using the `model.evaluate()` method provided by Keras.

Model evaluation serves several important purposes:

- **Generalization Check:** It measures how well the model performs on new, real-world-like data.
- **Performance Metrics:** It returns the **loss value** and any specified metrics (in this case, **accuracy**) for the test dataset.
- **Benchmarking:** It helps compare the performance of different models or configurations.

Test Dataset

The MNIST test dataset contains 10,000 images of handwritten digits that were **not used during training or validation**. Evaluating on this dataset provides an unbiased assessment of model accuracy.

Interpretation of Results

- A **high test accuracy** (e.g., above 97%) on the MNIST dataset typically indicates a well-performing model.
- A large **gap between training and test accuracy** might suggest **overfitting**, where the model memorized training data but failed to generalize.
- A **low test loss** indicates that the model's probability distributions for predictions are close to the true one-hot encoded labels.

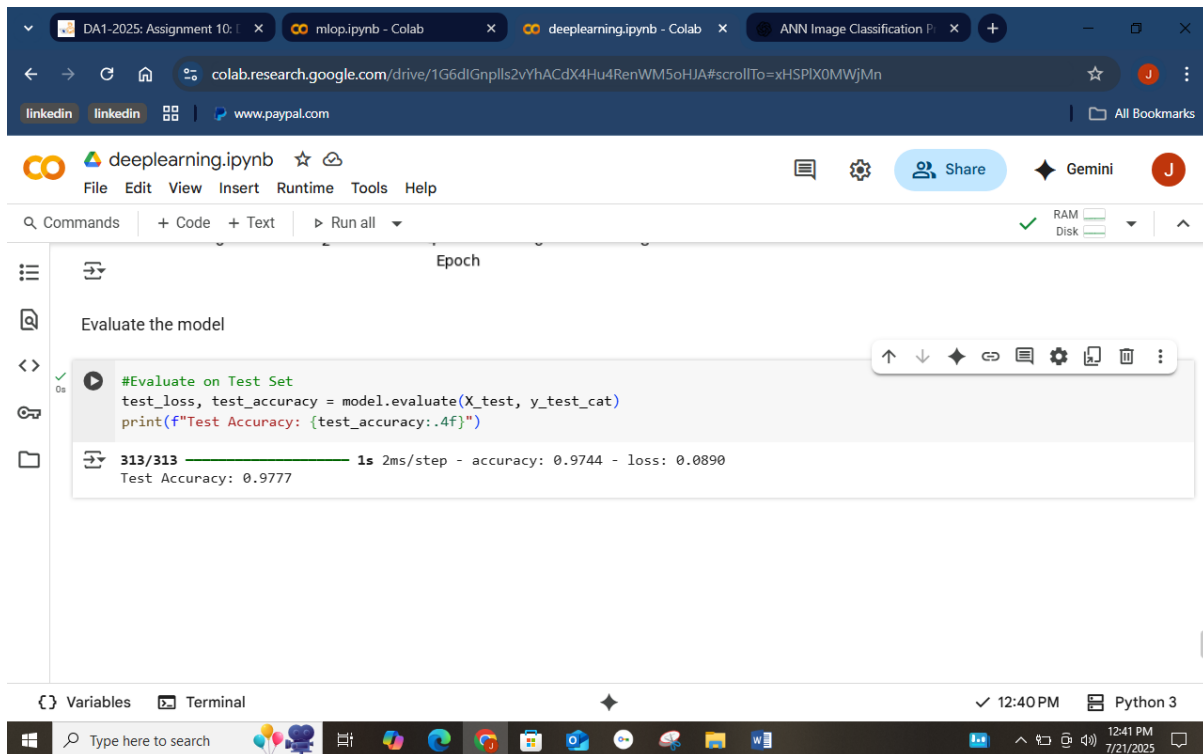


Fig 2.7.1: Output of evaluating the model

The model achieved a high test accuracy of **97.77%**, indicating excellent performance in classifying handwritten digits.

A low test loss of **0.0890** suggests the model's predictions are both accurate and confident.

The small gap between training and test accuracy implies good generalization with minimal overfitting.

Overall, the model is reliable and well-suited for MNIST digit classification tasks.

2.8: Making predictions and analysing performance

Further analysis was performed to understand model performance in more detail:

- **Predictions** were generated using `model.predict()` on the test set. These predictions were converted into class labels using `np.argmax`.
- A **confusion matrix** was generated using `sklearn.metrics.confusion_matrix`, and visualized with `seaborn.heatmap`. This shows the distribution of predicted vs. actual classes, highlighting which digits the model confuses most often.
- A **classification report** was printed using `sklearn.metrics.classification_report`, showing:
 - **Precision:** How many of the predicted digits were correct.
 - **Recall:** How many actual digits were correctly identified.
 - **F1-score:** The harmonic mean of precision and recall.

This detailed breakdown helps identify strengths and weaknesses in class-wise performance.

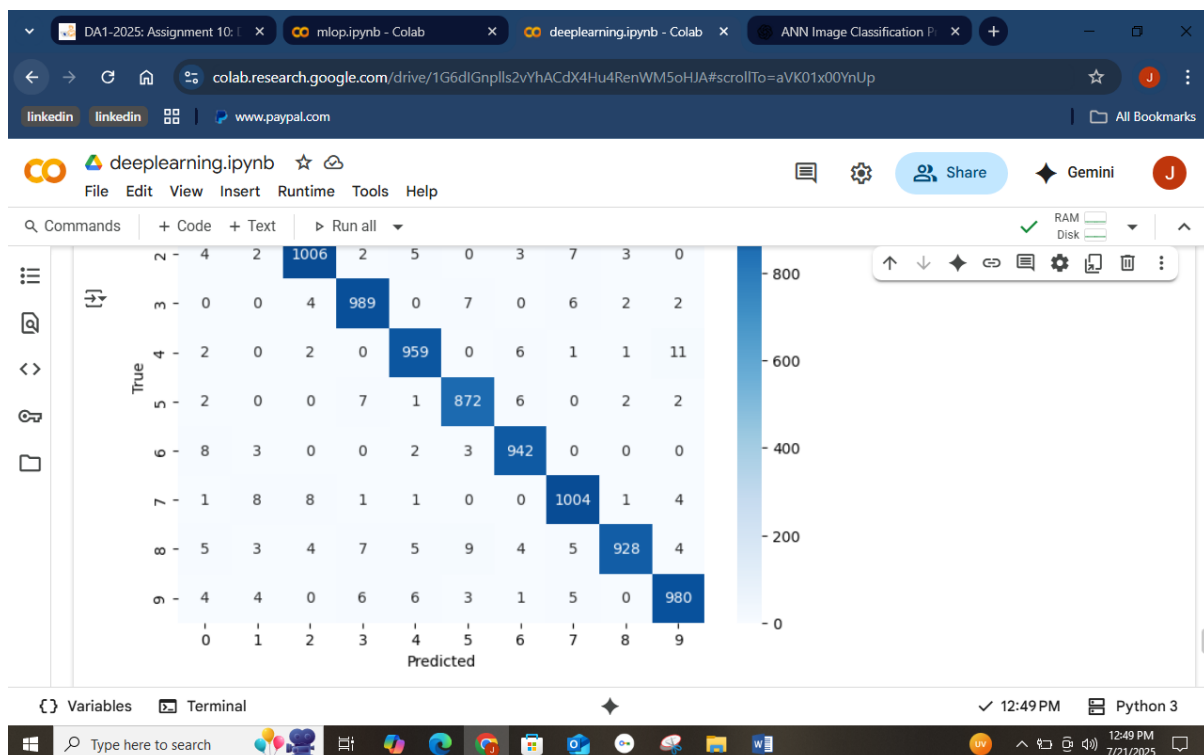


Fig 2.8.1: Output of predicted vs actual classes

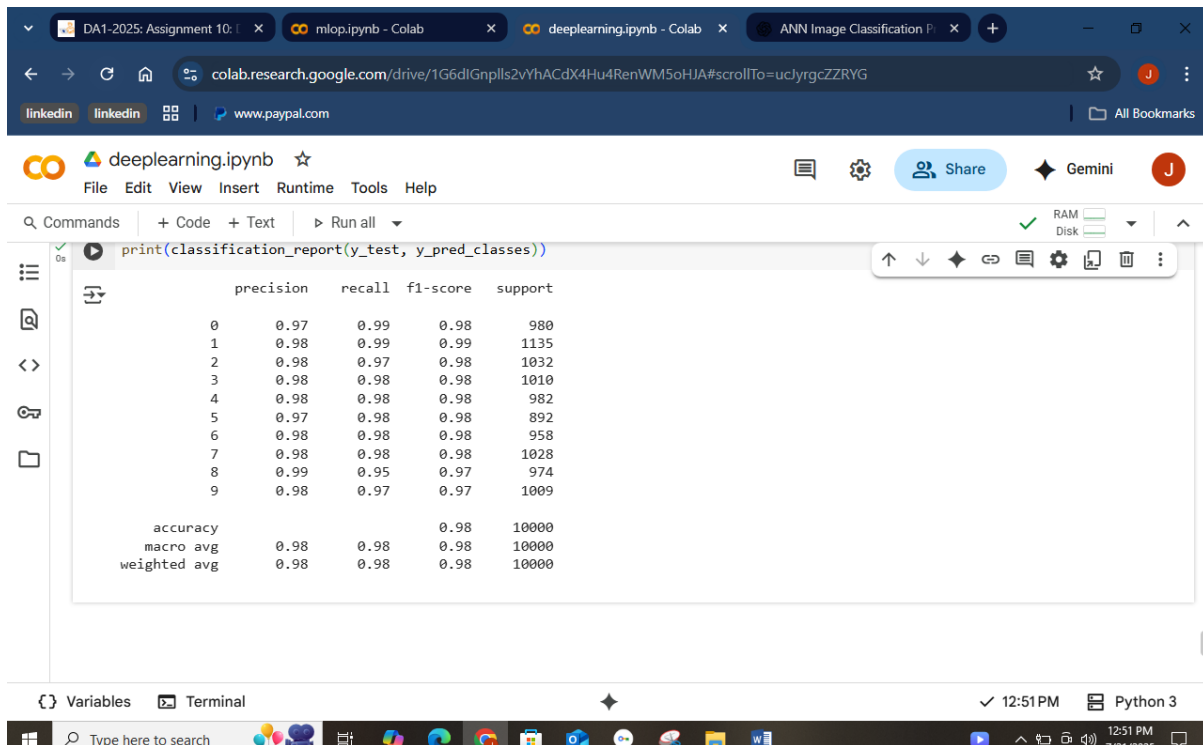


Fig 2.8.2:Output of classification report

2.9: Saving and Reloading the model

After successfully training and evaluating the neural network model, it is important to save it for future use whether for deployment, further training (fine-tuning) or simply preserving the model's current state. TensorFlow/Keras provides a straightforward way to do this using the `model.save()` function.

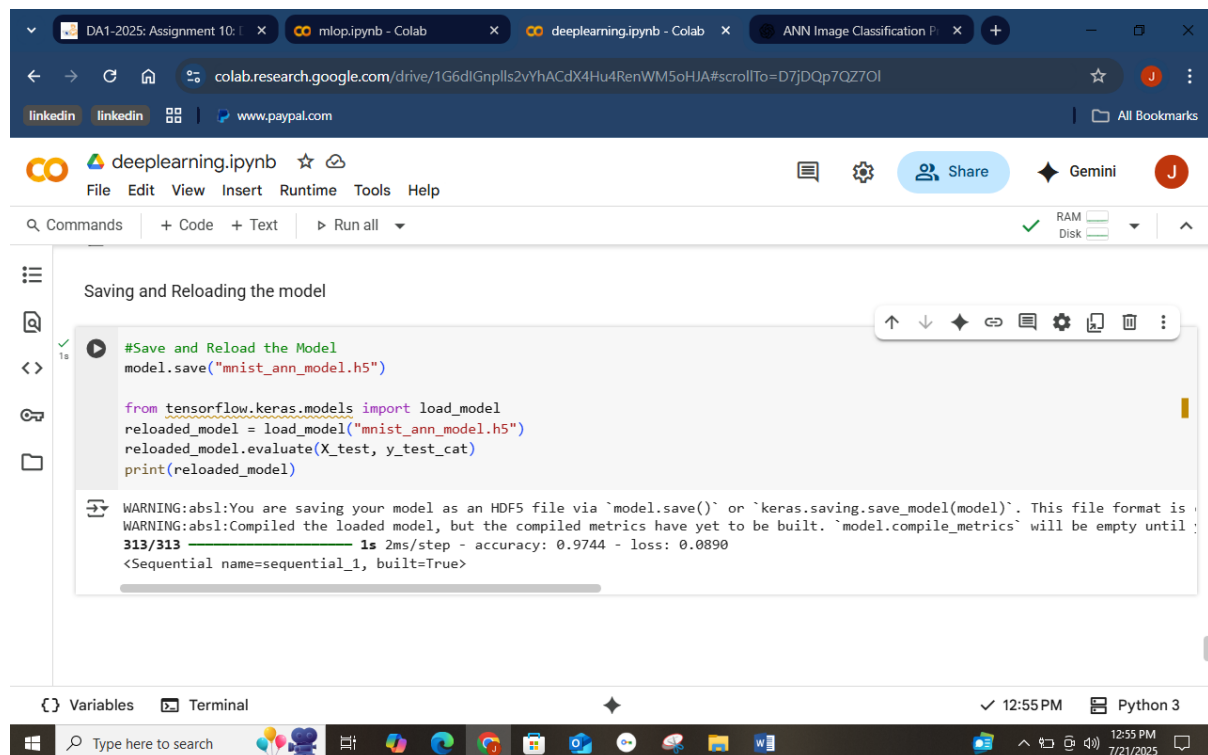
Saving the Model in Native Keras Format

In this project, the model is saved using the **modern Keras native format** (.keras), which stores the entire model architecture, weights, training configuration, and optimizer state in a single file. This format is highly recommended for current Keras projects.

The file contains:

- The model **architecture** (layers, activations, etc.)
- The model **weights** (learned parameters)
- The **training configuration** (loss function, optimizer, metrics)
- The **state of the optimizer**, allowing training to be resumed exactly from where it stopped.

Once loaded, `loaded_model` is ready for inference, evaluation, or further training just like the original model.



```
#Save and Reload the Model
model.save("mnist_ann_model.h5")

from tensorflow.keras.models import load_model
reloaded_model = load_model("mnist_ann_model.h5")
reloaded_model.evaluate(X_test, y_test_cat)
print(reloaded_model)
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is deprecated. We recommend you to use the native Keras format, which is supported by all Keras versions. To use the native Keras format, you can use `model.save('model.keras')` or `keras.saving.save\_model(model, 'model.keras')`.

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until `model.compile` is called.

313/313 ————— 1s 2ms/step - accuracy: 0.9744 - loss: 0.0890

<Sequential name=sequential_1, built=True>

Fig 2.9.1:Output of saving and reloading the model

Link to Code:

https://colab.research.google.com/drive/1G6dIGnpIls2vYhACdX4Hu4RenWM5oHJA?usp=s_haring

Conclusion

In this assignment, an Artificial Neural Network was successfully designed, implemented, and evaluated using the TensorFlow and Keras libraries to classify handwritten digits from the MNIST dataset. The project demonstrated the complete deep learning workflow — from data preprocessing and model design to training, evaluation, and saving the final model.

The model achieved strong performance on the test dataset, highlighting the effectiveness of neural networks in handling image classification tasks. Visualizations of the training history provided insights into model learning behavior, while saving and reloading the model reinforced practical skills in model deployment.

Overall, this project not only solidified foundational knowledge of neural networks and the TensorFlow/Keras framework but also showcased the power of deep learning in solving real-world classification problems efficiently and accurately.